LUCIDITY COWORK

Product & Technical Proposition

A Knowledge-Informed App Builder for the Browser

February 2026

Confidential

1. The Opportunity

What We're Building

A web-based AI workspace where users research, plan, and understand a project on a canvas, then an AI agent builds it inside a cloud sandbox — all in one browser tab. Think of it as Bolt.new with a brain, or Claude Cowork without the desktop limitation.

The Market Gap

The current market is split into three categories of tools, and none of them overlap:

Category	Products	What They Do	What They Lack
Thinkers	Lucidity, Notion AI	Help you understand and organize knowledge	Can't execute or build anything
Builders	Bolt.new, Lovable	Turn a prompt into a running web app	No understanding — build from blind one-line prompts
Doers	Claude Cowork	General desktop agent (files, research, tasks)	macOS only, stateless, no persistent knowledge

Lucidity Cowork sits in the center: understand first, then build. The canvas accumulates structured knowledge (data models, API docs, design references, competitor notes) and the agent builds from that rich context — not from a one-line prompt. This produces higher-quality first drafts and fewer iterations.

Why Now

- Claude Cowork is macOS-only desktop app — no web version exists. That's our window.
- Lovable hit $100M ARR in 8 months. Developers are spending $15B/year on AI coding tools. The demand is proven.
- Bolt/Lovable users burn tokens re-explaining context every edit. A knowledge layer eliminates this.
- Cloud sandbox infrastructure (E2B, Daytona) has matured — you can spin up an isolated Linux VM in 150ms via a 3-line API call.

2. Product Concept

The Core Loop: Think → Execute → Review

Every user session follows this cycle:

- **Think (Canvas):** User researches, threads ideas, pastes API docs, annotates design references, outlines data models. The canvas is Lucidity's existing strength. All of this becomes structured context in a knowledge graph.
- **Execute (Sandbox):** User says "build it." The agent reads the knowledge graph and generates code inside an isolated cloud VM. User sees a live preview of their app in an iframe alongside the canvas.
- **Review (Both):** User reviews the output, adds annotations, requests changes. The agent iterates with full context from the canvas. No re-explaining.

What the User Sees

A single browser tab, split into two panels:

- **Left panel — Knowledge Canvas:** Lucidity's existing threaded conversations, annotations, and research workspace. This is where thinking happens.
- **Right panel — Live Workspace:** Code editor (Monaco), terminal (xterm.js), and a live preview iframe showing the running app from the cloud sandbox.
- **Bottom bar — Agent Chat:** Conversational interface to the agent. Agent reads from canvas context, writes to sandbox. User steers with natural language.

Differentiation Summary

Dimension	Cowork	Bolt / Lovable	Lucidity Cowork
Platform	macOS desktop only	Browser (web app)	Browser (web app)
Memory	None across sessions	Project files only	Persistent knowledge graph
Starting Context	Cold prompt	One-line prompt	Rich canvas research
Live Preview	No	Yes	Yes (iframe)
Collaboration	Single user	Limited	Shareable workspaces
Deploy	Manual	One-click	One-click

3. Technical Architecture

System Overview

The system has four layers. Each is independently replaceable.

```
Browser (Next.js)
  ├─ Canvas UI (existing Lucidity frontend)
  ├─ Code Editor (Monaco) + Terminal (xterm.js)
  └─ Live Preview (iframe → sandbox URL)
        |
  LangGraph Orchestration Server (Python)
  ├─ Canvas Agent → understands, annotates, threads
  ├─ Executor Agent → writes code, runs commands
  └─ Review Agent → validates, suggests fixes
        |
  Knowledge Layer
  ├─ Supabase (Postgres + pgvector)
  └─ Redis (session cache)
        |
  Sandbox Layer
  └─ E2B (Firecracker microVMs, 1 per user session)
```

3a. Orchestration: LangGraph

Why LangGraph: Graph-based state machines map directly to the Think → Execute → Review loop. It has built-in checkpointing (critical for persistent sessions across page reloads), human-in-the-loop as a first-class concept (agent pauses and waits for user annotation), and it's model-agnostic (Gemini for canvas, Claude for execution). It benchmarks 2.2x faster than CrewAI.

How it works: Each user session is a LangGraph state machine. Nodes represent agent actions (read canvas, generate code, run in sandbox, review output). Edges represent transitions triggered by user input or agent decisions. State checkpoints persist to the database, so if a user closes their browser and returns tomorrow, the session resumes exactly where it left off.

Alternative considered: PydanticAI — lighter weight, more control, FastAPI-style. Good if you want to avoid framework abstractions. Would require building your own orchestration layer on top.

3b. Sandbox: E2B

What E2B is: An open-source (Apache 2.0) platform that gives each user session its own isolated Linux VM in the cloud, powered by Firecracker microVMs (the same tech behind AWS Lambda). Each sandbox boots in ~150ms, has full filesystem access, can run any language, install packages, start web servers, and expose preview URLs.

Why Firecracker over Docker: We're running untrusted AI-generated code. Containers share the host kernel — a kernel exploit in one container can reach others. Firecracker microVMs

have their own kernel with hardware-level isolation via KVM, but still boot in milliseconds with <5MB memory overhead. It's VM security at container speed.

Integration is minimal:

```
from e2b_code_interpreter import Sandbox

sandbox = Sandbox.create(template='nextjs-starter')
sandbox.filesystem.write('/app/page.tsx', agent_code)
sandbox.commands.run('cd /app && npm run dev', background=True)
preview_url = sandbox.get_host(3000)  # embed in iframe
```

Who else uses E2B: Perplexity (Pro data analysis), Manus (autonomous agent VMs), Hugging Face (DeepSeek-R1 replication), Groq (compound AI systems).

Pricing: $150/month base (Pro plan) + ~$0.05/hour per active sandbox (1 vCPU, 2GB RAM). At 1,000 users, sandbox cost is only ~$317/month — 5% of total infrastructure cost.

3c. Knowledge Layer: Supabase + pgvector

Supabase provides Postgres, auth, real-time subscriptions, and pgvector for embeddings — all in one managed service. This avoids needing a separate vector database (Pinecone is $70+/month) while keeping everything in SQL.

What gets stored: Canvas threads, annotations, research notes, data model outlines, and their vector embeddings. When the executor agent needs context, it queries pgvector for semantically relevant canvas content rather than dumping the entire canvas into the prompt.

Pricing: Free tier for MVP (500MB DB, 50K MAU). Pro at $25/month for production (8GB DB, 100K MAU).

3d. LLM Model Strategy

Two models serve different roles:

Role	Model	Why	Input Cost	Output Cost
Canvas Agent	Gemini 2.5 Flash	Fast, cheap, good at understanding. Free tier for MVP.	$0.30/1M tok	$2.50/1M tok
Executor Agent	Claude Sonnet 4.5	Best code generation. Reliable multi-step execution.	$3.00/1M tok	$15.00/1M tok
Simple Tasks	Claude Haiku 4.5	File ops, install cmds, routing. 3x cheaper.	$1.00/1M tok	$5.00/1M tok
Embeddings	text-embedding-3-small	Best cost/performance for knowledge graph.	$0.02/1M tok	N/A

4. Infrastructure Cost Summary

All assumptions are editable in the attached spreadsheet (lucidity_cowork_cost_model.xlsx). Key figures below assume default settings: 10–15 sessions/user/month, 30-minute sessions, 20 agent calls per session.

Monthly Cost by Phase

	MVP (20 users)	Growth (200)	Scale (1,000)
Sandbox (E2B)	$158	$200	$317
LLM APIs	$168	$1,346	$5,610
Database + Hosting + Misc	$6	$61	$135
TOTAL	**$332/mo**	**$1,607/mo**	**$6,062/mo**
Revenue ($25/mo, 10–20% conversion)	$50	$750	$5,000

The Critical Insight

LLM APIs are 92.5% of cost at scale. Sandboxes, databases, and hosting are negligible. The single biggest line item is Claude Sonnet output tokens at $3.60 per user per month (12 executor calls × 2,000 output tokens × $15/1M). All cost optimization efforts should focus on LLM usage.

Path to Profitability

Unoptimized, the model loses money at all stages. With these optimizations, Scale phase becomes profitable:

- **Prompt Caching:** Cache system prompts + knowledge context at 90% discount. Saves 40–60% on input tokens.
- **Model Routing:** Use Haiku ($1/$5) for simple tasks (file ops, installs, routing), Sonnet only for actual code generation. ~40% of calls are simple. Saves 30–50%.
- **Gemini Free Tier:** Google AI Studio is free (1,500 requests/day). Covers the entire canvas agent workload for MVP.
- **Auto-Sleep Sandboxes:** Kill idle sandboxes after 5 min of inactivity instead of the full session timeout. Saves 40–60% on compute.

With these applied, Scale cost drops from $6,062 to approximately $2,500–$3,000/month against $5,000 revenue — a positive gross margin of 40–50%.

5. Build Phases

Phase 1: MVP (Weeks 1–6)

Goal: One user can go from canvas research to a running web app in the browser.

- **Frontend:** Add a split-pane layout to the existing Lucidity canvas. Right pane has Monaco editor, xterm.js terminal, and an iframe for live preview.
- **Backend:** Set up a LangGraph orchestration server (Python, deployed on Railway/Fly.io). Two nodes: Canvas Agent (Gemini Flash) and Executor Agent (Claude Sonnet).
- **Sandbox:** Integrate E2B SDK. Create a custom sandbox template pre-loaded with Node.js, Python, and common frameworks. On “Build” click, spawn sandbox, pipe agent output to it, return preview URL to iframe.
- **Knowledge:** Supabase Free tier. Store canvas threads and annotations in Postgres. Enable pgvector extension for embedding-based retrieval.
- **Auth:** Supabase Auth (email + Google OAuth). Free tier.

Estimated cost: ~$332/month for 20 test users. Mostly E2B base fee + LLM tokens.

Phase 2: Growth (Months 3–9)

Goal: 200 users, paid subscriptions, reliable multi-session persistence.

- **Persistence:** Implement E2B sandbox pause/resume so users can close the browser and return to their project. Alternatively evaluate Daytona for native persistence.
- **LLM Optimization:** Implement prompt caching, model routing (Haiku for simple tasks), and structured output format to reduce token usage.
- **Deployment:** One-click deploy to Vercel/Netlify from the sandbox. User gets a production URL.
- **Billing:** Stripe integration. $25/month Starter plan (100 agent sessions). Free tier (5 sessions/day).
- **Upgrade infra:** Supabase Pro ($25/mo), Vercel Pro ($20/mo), Sentry for error tracking.

Phase 3: Scale (Months 9–18)

Goal: 1,000+ users, sustainable unit economics, team features.

- **Collaboration:** Shared workspaces where teams can co-annotate the canvas and share sandbox environments.
- **Self-host sandboxes:** Evaluate migrating from E2B managed to self-hosted Firecracker or microsandbox if concurrent sandbox count exceeds 100+. Potential 50–70% cost reduction.
- **Advanced knowledge graph:** Richer RAG with re-ranking, cross-project knowledge sharing, user preference learning.

- **Template marketplace:** Pre-built project templates (SaaS starter, e-commerce, dashboard) that agents scaffold from.

6. Key Technical Decisions for the Developer

Decision 1: WebSocket Architecture

The frontend needs real-time bidirectional communication with the sandbox (terminal output, file changes, agent streaming). Use WebSockets from the browser to the LangGraph server, which proxies to the E2B sandbox. This means the orchestration server is stateful per session — plan for sticky sessions or use LangGraph's built-in checkpointing to handle reconnects.

Decision 2: Canvas → Knowledge Graph Schema

This is the most important design decision and needs to be defined early. Every canvas thread, annotation, and conclusion needs to be stored as a structured node with:

- Type (research, data-model, design-reference, api-doc, decision, requirement)
- Content (text + optional file attachments)
- Vector embedding (for semantic retrieval by the agent)
- Relationships (this annotation replies to that thread, this decision depends on that research)

The executor agent queries this graph before generating code. The quality of code output is directly proportional to how well this schema captures the user's intent.

Decision 3: Human-in-the-Loop Interrupt Points

The agent should NOT run autonomously for 50 steps. Define explicit interrupt points in the LangGraph state machine where the agent pauses and asks the user to review:

- After scaffolding the project structure (before writing implementation code)
- After generating the data model / schema
- After completing each major feature
- When the agent encounters ambiguity it can't resolve from the canvas context

These interrupts map to LangGraph's interrupt() function and should surface in the UI as a review prompt with the agent's proposed plan.

Decision 4: Sandbox Template Strategy

Create 3–5 pre-built E2B templates to avoid cold-start package installation:

- nextjs-starter: Node 20, Next.js 15, Tailwind, Prisma, pre-installed
- flask-starter: Python 3.12, Flask, SQLAlchemy, common data science libs
- fullstack-starter: Node + Python, both runtimes available

Templates are built via Dockerfile-like configs and cached as snapshots. Users get a ready-to-code environment in 150ms instead of waiting 30–60 seconds for npm install.

7. Tech Stack Summary

Layer	Technology	Why This Choice
Frontend	Next.js + existing Lucidity UI	Existing codebase. Add Monaco, xterm.js, split panes.
Orchestration	LangGraph (Python)	Graph state machines, checkpointing, human-in-the-loop, model-agnostic.
Sandbox	E2B (Firecracker microVMs)	150ms boot, hardware isolation, preview URLs, Apache 2.0, battle-tested.
Database	Supabase (Postgres + pgvector)	DB + vector store + auth + real-time in one. Free tier for MVP.
Cache	Upstash Redis	Serverless Redis. Session state, agent cache. Free tier.
LLM (Canvas)	Gemini 2.5 Flash	Cheap, fast, free tier for MVP. Good at understanding/annotation.
LLM (Executor)	Claude Sonnet 4.5	Best code generation. $3/$15 per 1M tokens.
LLM (Simple)	Claude Haiku 4.5	File ops, routing, installs. $1/$5 per 1M tokens.
Embeddings	OpenAI text-embedding-3-small	$0.02/1M tokens. Best cost/performance.
Hosting (Frontend)	Vercel	Free hobby tier. Pro $20/mo when needed.
Hosting (Backend)	Railway or Fly.io	LangGraph server + WebSocket support. $5–25/mo.
Payments	Stripe	Subscription billing. Standard 2.9% + $0.30.

8. First Week: What to Build First

The fastest path to a working prototype:

- **Day 1–2:** Get E2B working. Create an account, get an API key, write a Python script that spins up a sandbox, writes a Next.js app, starts the dev server, and returns a preview URL. Prove the sandbox loop works.
- **Day 3–4:** Set up LangGraph. Build a minimal two-node graph: one node calls Gemini Flash with canvas context and produces a plan, the other calls Claude Sonnet to generate code and writes it to E2B. Hard-code some sample canvas context for now.
- **Day 5–6:** Connect to the UI. Add a split-pane layout to Lucidity. Left pane is the existing canvas. Right pane is an iframe that loads the E2B preview URL. Add a “Build” button that triggers the LangGraph pipeline and streams agent output.
- **Day 7:** Demo. A user types research notes on the canvas, hits “Build,” and sees a live web app appear in the right pane. That’s your MVP proof-of-concept.

— End of Proposition —